

Dipartimento di Ingegneria dell'Informazione
Facoltà di Ingegneria

Sistemi Operativi: Raccolta di Esercizi

prof. Luca Spalazzi

a.a. 2012/2013

ESERCIZIO 1

Struttura di processo che genera un duplicato di se stesso e NON attende la terminazione del figlio.

```
#include <stdio.h>
void main (int argc, char *argv[])
{
    pid = fork();                /* genera nuovo processo */
    if (pid < 0) {                /* errore */
        fprintf(stderr, "creazione nuovo processo fallita");
        exit(-1);
    }
    else if (pid == 0) { /* processo figlio */
        /* istruzioni del processo figlio */;
    }
    else {                      /* processo padre */
        /* istruzioni del processo padre */
        exit(0);
    }
}
```

ESERCIZIO 2

Processo per ordinare un vettore di 100 elementi, il padre ordina i primi 50 elementi, il figlio ordina gli altri 50. Senza memoria condivisa NON funziona.

```
#include <stdio.h>
int vett[100];
void main (int argc, char *argv[])
{
    pid = fork();                /* genera nuovo processo */
    if (pid < 0) {                /* errore */
        fprintf(stderr, "creazione nuovo processo fallita");
        exit(-1);
    }
    else if (pid == 0) { /* processo figlio */
        sort(vett, 50, 99);
    }
    else { /* processo padre */
        sort(vett, 0, 49);
    }
    merge(vett, 0, 50, 99);
    exit(0);
}

void sort (int vett[], inizio, fine)
{
    /* procedura di ordinamento */
}

void merge (int vett[], inizio, meta, fine)
{
    /* procedura di fusione */
}
```

ESERCIZIO 3

Processo multithread per ordinare un vettore di 100 elementi, il thread padre genera due thread figlio: il primo ordina i primi 50 elementi, il secondo ordina gli altri 50. I thread hanno memoria condivisa e quindi funziona.

```
#include <pthread.h>
#include <stdio.h>
int vett[100];          /* questo vettore è condiviso dai thread */
void *sort( int vett[], inizio, fine ) /* il thread */

void main (int argc, char *argv[])
{
    pthread_t tid1, tid2; /* identificatore del thread */
    pthread_attr_t attr;  /* attributi del thread */
    if (argc != 2) || (atoi(argv[1]) < 0) /* errore */
        exit(-1);
    else {
        /* reperisce gli attributi predefiniti */
        pthread_attr_init(&attr);
        /* creazione del thread */
        pthread_create(&tid1, &attr, sort, vett, 0, 49);
        pthread_create(&tid2, &attr, sort, vett, 50, 99);
        /* attende la terminazione dei thread */
        pthread_join(tid1, NULL);
        pthread_join(tid2, NULL);
        merge(vett, 0, 50, 99);
    }
}

void *sort (int vett[], inizio, fine)
{
    /* procedura di ordinamento */
    ...
    pthread_exit(0); /* terminazione del thread */
}

void merge (int vett[], inizio, meta, fine)
{
    /* procedura di fusione */
}
```

ESERCIZIO 4

Problema del produttore-consumatore a buffer limitato senza utilizzare la variabile counter ma utilizzando solo le variabili in e out. Questa soluzione ha il problema che in e out crescono all'infinito e quindi si incorre in problemi di overflow.

```
#define BUFFER_SIZE 10
typedef struct {
    . . .
} item;
item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;

item nextProduced;
*while (1) {
    while ( (in - out) == BUFFER_SIZE)
        ; /* do nothing */
    buffer[in % BUFFER_SIZE] = nextProduced;
    in = in + 1;
}

item nextConsumed;
while (1) {
    while ( (in - out) == 0)
        ; /* do nothing */
    nextConsumed = buffer[out % BUFFER_SIZE];
    out = out + 1;
}
```

Come il precedente ma senza problemi di overflow.
VERIFICARE SE FUNZIONA!!!

```
#define BUFFER_SIZE 10
typedef struct {
    . . .
} item;
item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;

item nextProduced;
*while (1) {
    while ( (in - out) == BUFFER_SIZE)
        ; /* do nothing */
    buffer[in % BUFFER_SIZE] = nextProduced;
    in = (in + 1) % (2*BUFFER_SIZE);
}

item nextConsumed;
while (1) {
    while ( (in - out) == 0)
        ; /* do nothing */
    nextConsumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
}
```

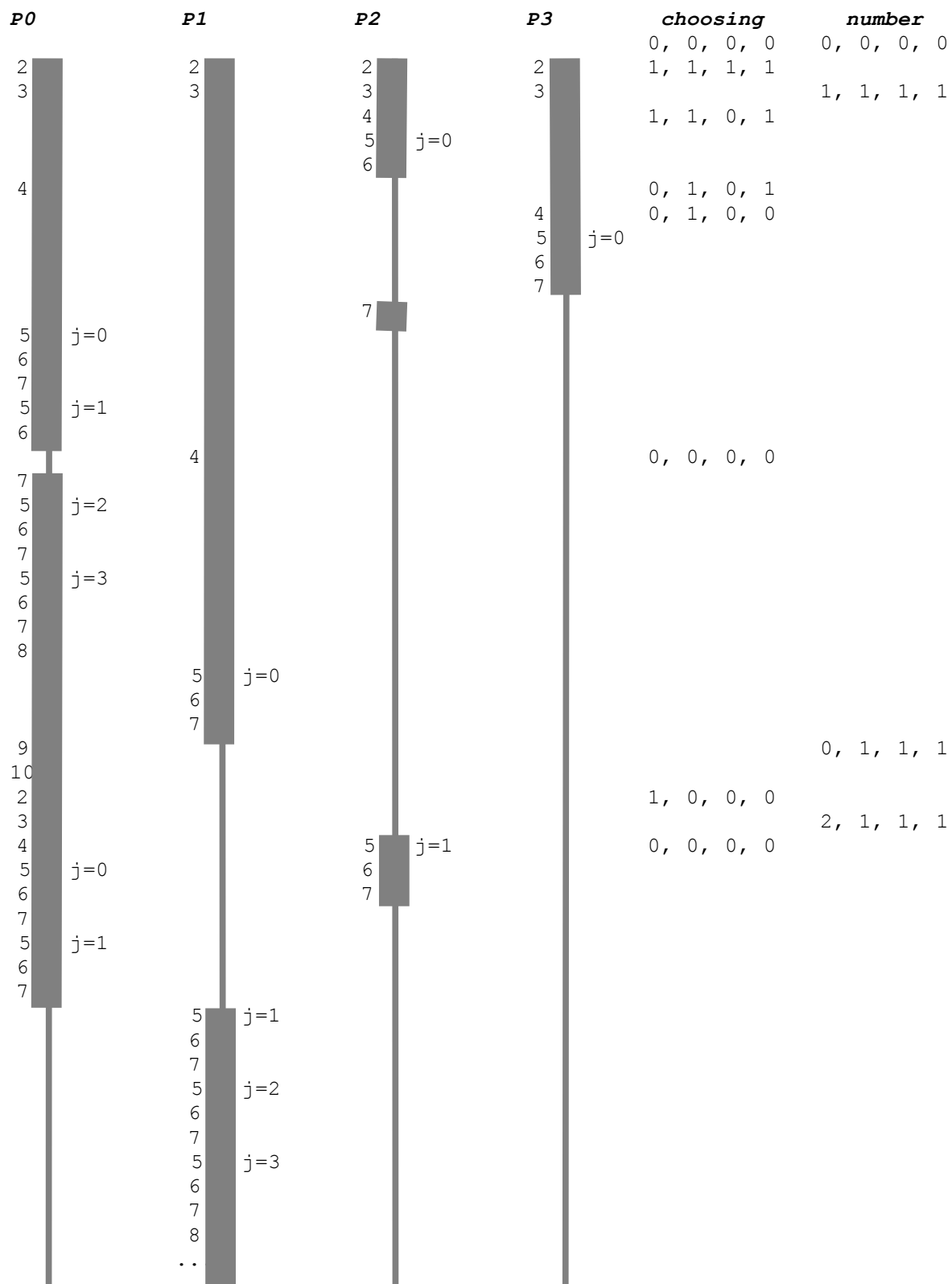
ESERCIZIO 5

Algoritmo del Fornaio: esempio di funzionamento nel caso con 4 processi

```
boolean choosing[4] = (0 , 0 , 0 , 0);
```

```
int number[4]      = (0 , 0 , 0 , 0);
```

```
1  do {
2    choosing[i] = true;
3    number[i] = max(number[0],number[1],number[2],number[3])+1;
4    choosing[i] = false;
5    for (j = 0; j < 4; j++) {
6      while (choosing[j]) ;
7      while ( (number[j] != 0) &&
              ((number[j],j) < (number[i],i))) ;
      }
8    critical section
9    number[i] = 0;
10   remainder section
11 } while (1);
```



ESERCIZIO 6

Soluzione regione critica con TSL. Dire se soddisfa le proprietà di mutua esclusione, progresso e attesa limitata.

```
boolean waiting[n] = (0 , 0 , ... , 0);
```

```
boolean lock       = 0;
```

```
1 do {
2   waiting[i] = true;
3   key = true;
4   while (waiting[i] && key)
5     key = TestAndSet(&lock);
6   critical section
7   j = (i+ 1) % n;
8   while ( (j!=i) && !waiting[i])
9     j = (i+ 1) % n;
10  if (j==i)
11    lock = false;
12  else
13    waiting[j] = false;
14  remainder section
15} while (1);
```

ESERCIZIO 7

Realizzazione di un semaforo con contatore con un semaforo binario

Data structures:

```
binary-semaphore S1, S2;  
int C;
```

Initialization:

```
S1 = 1  
S2 = 0  
C = initial value of semaphore S
```

wait operation

```
wait(S1);  
C--;  
if (C < 0) {  
    signal(S1);  
    wait(S2);  
}  
signal(S1);
```

signal operation

```
wait(S1);  
C ++;  
if (C <= 0)  
    signal(S2);  
else  
    signal(S1);
```

ESERCIZIO 8

Soluzione dei 5 filosofi dove i filosofi di ordine pari chiedono prima il bastoncino di sinistra mentre quelli di ordine dispari chiedono prima quella di destra. Dire se soddisfa le proprietà di mutua esclusione, progresso e attesa limitata.

```
semaphore fork[5] = (1,1,1,1,1);
```

```
philosopher(int i) {  
    while(TRUE) {  
        think();  
        j = i % 2;  
        wait(fork[(i+j) mod 5]);  
        wait(fork[(i+1-j) mod 5]);  
        eat();  
        signal(fork[(i+1-j) mod 5]);  
        signal(fork[(i+j) mod 5]);  
    }  
}
```

ESERCIZIO 9

Soluzione dei 5 filosofi con semafori privati, i semafori sono inizializzati a falso. Dire se soddisfa le proprietà di mutua esclusione, progresso e attesa limitata.

```
#define N          5                /* number of philosophers */
#define LEFT      (i+N-1)%N        /* number of i's left neighbor */
#define RIGHT     (i+1)%N          /* number of i's right neighbor */
#define THINKING  0                /* philosopher is thinking */
#define HUNGRY    1                /* philosopher is trying to get forks */
#define EATING    2                /* philosopher is eating */
typedef int semaphore;             /* semaphores are a special kind of int */
int state[N];                     /* array to keep track of everyone's state */
semaphore mutex = 1;              /* mutual exclusion for critical regions */
semaphore s[N];                   /* one semaphore per philosopher */

void philosopher(int i)            /* i: philosopher number, from 0 to N-1 */
{
    while (TRUE) {                 /* repeat forever */
        think();                   /* philosopher is thinking */
        take_forks(i);             /* acquire two forks or block */
        eat();                     /* yum-yum, spaghetti */
        put_forks(i);              /* put both forks back on table */
    }
}

void take_forks(int i)             /* i: philosopher number, from 0 to N-1 */
{
    down(&mutex);                  /* enter critical region */
    state[i] = HUNGRY;             /* record fact that philosopher i is hungry */
    test(i);                       /* try to acquire 2 forks */
    up(&mutex);                    /* exit critical region */
    down(&s[i]);                   /* block if forks were not acquired */
}

void put_forks(i)                  /* i: philosopher number, from 0 to N-1 */
{
    down(&mutex);                  /* enter critical region */
    state[i] = THINKING;           /* philosopher has finished eating */
    test(LEFT);                   /* see if left neighbor can now eat */
    test(RIGHT);                  /* see if right neighbor can now eat */
    up(&mutex);                   /* exit critical region */
}

void test(i)                       /* i: philosopher number, from 0 to N-1 */
{
    if (state[i] == HUNGRY && state[LEFT] != EATING && state[RIGHT] != EATING) {
        state[i] = EATING;
        up(&s[i]);
    }
}

#define N          5                /* number of philosophers */
#define LEFT      (i+N-1)%N        /* number of i's left neighbor */
#define RIGHT     (i+1)%N          /* number of i's right neighbor */
#define THINKING  0                /* philosopher is thinking */
#define HUNGRY    1                /* philosopher is trying to get forks */
#define EATING    2                /* philosopher is eating */
typedef int semaphore;             /* semaphores are a special kind of int */
int state[N];                     /* array to keep track of everyone's state */
semaphore mutex = 1;              /* mutual exclusion for critical regions */
semaphore s[N];                   /* one semaphore per philosopher */

void philosopher(int i)            /* i: philosopher number, from 0 to N-1 */
{
    while (TRUE) {                 /* repeat forever */
        think();                   /* philosopher is thinking */
        take_forks(i);             /* acquire two forks or block */
        eat();                     /* yum-yum, spaghetti */
        put_forks(i);              /* put both forks back on table */
    }
}

void take_forks(int i)             /* i: philosopher number, from 0 to N-1 */
{
    down(&mutex);                  /* enter critical region */
    state[i] = HUNGRY;             /* record fact that philosopher i is hungry */
    test(i);                       /* try to acquire 2 forks */
    up(&mutex);                    /* exit critical region */
    down(&s[i]);                   /* block if forks were not acquired */
}

void put_forks(i)                  /* i: philosopher number, from 0 to N-1 */
{
    down(&mutex);                  /* enter critical region */
    state[i] = THINKING;           /* philosopher has finished eating */
    test(LEFT);                   /* see if left neighbor can now eat */
    test(RIGHT);                  /* see if right neighbor can now eat */
    up(&mutex);                   /* exit critical region */
}

void test(i)                       /* i: philosopher number, from 0 to N-1 */
{
    if (state[i] == HUNGRY && state[LEFT] != EATING && state[RIGHT] != EATING) {
        state[i] = EATING;
        up(&s[i]);
    }
}
```

ESERCIZIO 10

Supponete che siano arrivati, nell'ordine, i processi P1, P2, P3, P4. La seguente tabella riporta: a) l'istante di arrivo dei processi nella coda dei processi pronti; b) la stima dell'ultimo CPU burst (τ_n); c) la durata effettiva dell'ultimo CPU burst (t_n); d) la durata effettiva del CPU burst successivo (t_{n+1}).

Processo	Istante d'arrivo	τ_n	t_n	t_{n+1}
P1	0	16	14	13
P2	5	6	4	6
P3	7	2	5	3
P4	8	3	3	4

Calcolate la stima del CPU burst successivo supponendo che il peso α della media esponenziale sia pari a 0.5. Quale è il diagramma di Gantt corretto se è stato applicato l'algoritmo di scheduling SJF con prelazione (utilizzare la media esponenziale τ_{n+1} come stima del CPU burst successivo t_{n+1})?

ESERCIZIO 11

Supponete che siano arrivati, nell'ordine, i processi P1, P2, P3, P4. La seguente tabella riporta: a) l'istante di arrivo dei processi nella coda dei processi pronti; b) la durata effettiva del CPU burst successivo (t_{n+1}). c) la classe di scheduling, d) la priorità real-time, e) il fattore di nice, f) il quanto di tempo per processi in classe real-time round robin

Processo	Istante d'arrivo	t_{n+1}
P1	0	19
P2	7	5
P3	11	14
P4	13	4

Quale è il diagramma di Gantt corretto se si suppone di utilizzare un algoritmo di scheduling a code multiple con feedback organizzato su tre livelli: il primo livello adotta un round robin (RR) con quanto di tempo pari a 5, il secondo livello adotta un round robin (RR) con quanto di tempo pari a 10, il terzo livello adotta un FCFS.

ESERCIZIO 12

Dati quattro processi P1, P2, P3 e P4 con la seguente struttura: hanno un primo CPU burst, seguito da un I/O burst (accesso ad un dato cilindro del disco DSK), seguito da un secondo CPU burst, poi terminano. La seguente tabella riporta: a) l'istante in cui i processi sono inseriti per la prima volta nella coda dei processi pronti; b) la stima della durata del 1° CPU burst; c) la durata effettiva del 1° CPU burst; la priorità dei processi; d) il cilindro a cui devono accedere durante l'I/O burst; e) la durata effettiva del 2° CPU burst.

Processo	Tempo di arrivo	1° CPU burst Stima (τ_1)	1° CPU burst Durata effettiva (t_1)	I/O burst n° cilindro	Priorità	2° CPU burst Durata effettiva (t_2)
P1	0	9	4	400	4	7
P2	2	5	7	600	3	11
P3	3	3	6	200	2	11
P4	5	7	4	800	1	7

Dire quali sono i diagrammi di Gantt corretti supponendo di utilizzare i seguenti algoritmi di scheduling:

versione a)

- CPU: scheduling FCFS.

- DSK: scheduling SSTF. Tenete presente che i cilindri sono numerati da 0 fino a 950, la testina inizialmente è posizionata nel cilindro 150 e si muove nella direzione crescente. Inoltre, approssimate la durata dell'I/O burst con il seek time, supponendo che la testina si muova con una velocità di 0,1 ms per cilindro.

versione b)

- CPU: scheduling FCFS.

- DSK: scheduling SCAN. Tenete presente che i cilindri sono numerati da 0 fino a 950, la testina inizialmente è posizionata nel cilindro 150 e si muove nella direzione crescente. Inoltre, approssimate la durata dell'I/O burst con il seek time, supponendo che la testina si muova con una velocità di 0,1 ms per cilindro.

versione c)

- CPU: scheduling FCFS.

- DSK: scheduling LOOK. Tenete presente che i cilindri sono numerati da 0 fino a 950, la testina inizialmente è posizionata nel cilindro 150 e si muove nella direzione crescente. Inoltre, approssimate la durata dell'I/O burst con il seek time, supponendo che la testina si muova con una velocità di 0,1 ms per cilindro

versione d)

- CPU: scheduling SJF con prelazione. Tenete presente che dovete utilizzare la media esponenziale (con α pari a 0.5) come stima dei CPU burst.

- DSK: scheduling SSTF. Tenete presente che i cilindri sono numerati da 0 fino a 950, la testina inizialmente è posizionata nel cilindro 150 e si muove nella direzione crescente. Inoltre, approssimate la durata dell'I/O burst con il seek time, supponendo che la testina si muova con una velocità di 0,1 ms per cilindro

versione e)

- CPU: scheduling SJF con prelazione. Tenete presente che dovete utilizzare la media esponenziale (con α pari a 0.5) come stima dei CPU burst.

- DSK: scheduling SCAN. Tenete presente che i cilindri sono numerati da 0 fino a 950, la testina inizialmente è posizionata nel cilindro 150 e si muove nella direzione crescente. Inoltre, approssimate la durata dell'I/O burst con il seek time, supponendo che la testina si muova con una velocità di 0,1 ms per cilindro

versione f)

- CPU: scheduling per priorità con prelazione e con invecchiamento (aging). Per l'invecchiamento tenete presente che per ogni 10 ms di attesa nella coda dei processi pronti la priorità un processo viene alzata di 1 (il numero della priorità viene decrementato di 1).
- DSK: scheduling LOOK. Tenete presente che i cilindri sono numerati da 0 fino a 950, la testina inizialmente è posizionata nel cilindro 150 e si muove nella direzione crescente. Inoltre, approssimate la durata dell'I/O burst con il seek time, supponendo che la testina si muova con una velocità di 0,1 ms per cilindro

versione g)

- CPU: scheduling Round Robin con quanto di tempo pari a 5.
- DSK: scheduling SSTF. Tenete presente che i cilindri sono numerati da 0 fino a 950, la testina inizialmente è posizionata nel cilindro 150 e si muove nella direzione crescente. Inoltre, approssimate la durata dell'I/O burst con il seek time, supponendo che la testina si muova con una velocità di 0,1 ms per cilindro

versione g)

- CPU: scheduling Round Robin con quanto di tempo pari a 5.
- DSK: scheduling SCAN. Tenete presente che i cilindri sono numerati da 0 fino a 950, la testina inizialmente è posizionata nel cilindro 150 e si muove nella direzione crescente. Inoltre, approssimate la durata dell'I/O burst con il seek time, supponendo che la testina si muova con una velocità di 0,1 ms per cilindro

versione g)

- CPU: scheduling Round Robin con quanto di tempo pari a 5.
- DSK: scheduling LOOK. Tenete presente che i cilindri sono numerati da 0 fino a 950, la testina inizialmente è posizionata nel cilindro 150 e si muove nella direzione crescente. Inoltre, approssimate la durata dell'I/O burst con il seek time, supponendo che la testina si muova con una velocità di 0,1 ms per cilindro

ESERCIZIO 13

Supponete di utilizzare la allocazione contigua a partizioni variabili, supponete che in memoria siano presenti i seguenti spazi liberi:

500K, 150K, 400K, 200K, 350K.

Come verrebbero allocati i seguenti processi:

P1 (180K), P2 (340K), P3 (300K), P4 (410K)

Se viene adottato:

- a) l'algoritmo First-fit?
- b) l'algoritmo Best-fit?
- c) l'algoritmo Worst-fit?

ESERCIZIO 14

Supponete di avere frame di 4K e la seguente tabella delle pagine:

Pagina	Frame
0	8192
1	0000
2	12288
3	4096

Quale è l'indirizzo fisico dei seguenti indirizzi logici? Tenete presente che gli indirizzi logici sono espressi nel formato <numero di pagina - scostamento>

- a) 3-3840 b) 2-1024 c) 0-4098

Risposta:

- a) 7936 b) 13312 c) trap: perché l'indirizzo esce dal frame assegnato alla pagina 0

ESERCIZIO 15

Supponete di avere la seguente tabella dei segmenti:

Segmento	Base	Limite
0	4096	2200
1	12288	780
2	0000	1000
3	8192	440

Quale è l'indirizzo fisico dei seguenti indirizzi logici? Tenete presente che gli indirizzi logici sono espressi nel formato <numero di segmento - scostamento>

- a) 0-2220 b) 1-720 c) 3-410

Risposta:

- a) trap b) 13008 c) 8602

ESERCIZIO 16

Supponete che un dato sistema operativo utilizzi la memoria virtuale. Supponete che la successione dei riferimenti alle pagine di un dato processo, a partire da un certo istante, sia la seguente:

3, 0, 3, 4, 7, 1, 1, 4, 2, 3, 4, 5, 6, 7, 1, 0, 0

Supponete che i frame a disposizione del processo siano 4. Calcolate il numero di page fault nel caso venga adottato per la sostituzione delle pagine:

- a) l'algoritmo FIFO
- b) l'algoritmo Ottimo
- c) l'algoritmo LRU (realizzato ad esempio con uno stack)
- d) l'algoritmo con bit di riferimento
- e) l'algoritmo dell'orologio
- f) l'algoritmo basato sulla doppia lista (pagine attive e inattive) tenendo conto che ogni 5 riferimenti viene spostata una pagina dalla lista delle pagine attive alla lista delle pagine inattive

ESERCIZIO 17

Supponete che un dato sistema operativo utilizzi la memoria virtuale. Supponete che la successione dei riferimenti alle pagine di un dato processo, a partire da un certo istante, sia la seguente:

3, 0, 3, 4, 7, 1, 1, 4, 2, 3, 4, 5, 6, 7, 1, 0, 0

Supponete che i frame a disposizione del processo siano 4 e il Δ pari a 5.

- a) Calcolate l'evoluzione temporale del working set.
- b) Dite se il sistema è a rischio di paginazione degenerare.

ESERCIZIO 18

Supponete di avere i seguenti processi:

Processo	Pagine	Priorità
P1	10	1
P2	15	2
P3	20	3
P4	35	4

La colonna Pagine indica la dimensione del processo in pagine, la colonna Priorità indica la priorità del processo (numero minore corrisponde a priorità maggiore). Supponete che in memoria ci siano 40 frame liberi. Calcolate quanti frame vengono assegnati a ciascun processo nel caso venga adottato:

- a) l'algoritmo di assegnazione uniforme,
- b) l'algoritmo di assegnazione proporzionale,
- c) l'algoritmo di assegnazione per priorità.

ESERCIZIO 19

Supponete di utilizzare per la allocazione delle pagine l'algoritmo Buddy-heap, supponete che inizialmente in memoria ci siano 64 pagine libere contigue.

Come verrebbe gestita la seguente successione di richieste?

- P1 richiede 8 pagine,
- P2 richiede 4 pagine,
- P3 richiede 16 pagine,
- P2 rilascia 4 pagine,
- P4 richiede 16 pagine,
- P1 rilascia 8 pagine.

ESERCIZIO 20

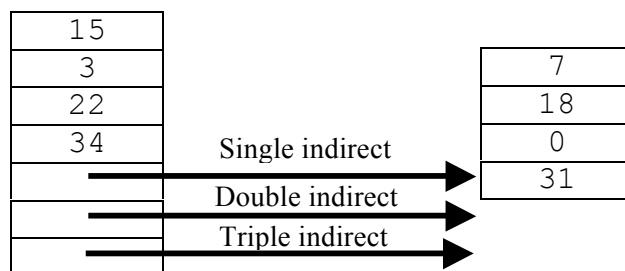
Supponete di avere un file composto da 100 blocchi (numerati da 0 a 99). Supponete che il file control block sia già in memoria. Supponete di dover aggiungere un nuovo blocco (presente in memoria centrale) e che ci sia spazio solo alla fine del file oppure di dover rimuovere un blocco. Quante operazioni di lettura/scrittura sul disco sono necessarie per ottenere le seguenti situazioni:

- a) Il blocco viene aggiunto all'inizio.
- b) Il blocco viene aggiunto dopo il blocco 50
- c) Il blocco viene aggiunto alla fine
- d) Viene rimosso il primo blocco.
- e) Viene rimosso il blocco 50
- f) Viene rimosso l'ultimo blocco

nel caso in cui si utilizza l'assegnazione contigua, concatenata, FAT, o indicizzata.

ESERCIZIO 21

Supponete di utilizzare per i file l'assegnazione indicizzata e di avere un file con la seguente tabella indice (organizzata secondo uno schema combinato tipo quello di Unix).



Se ogni blocco è pari a 512 byte, quale è l'indirizzo fisico dei seguenti indirizzi logici? Tenete presente che l'indirizzo fisico è espresso nel formato <numero blocco - scostamento> e che tutti gli indirizzi (logici, numero blocco, scostamento) partono da zero.

- a) 718.
- b) 3300

ESERCIZIO 22

Supponete di avere il seguente file system di tipo FAT (in figura è riportata la FAT)

00000	
01000	15002
01001	09998
01002	03460
01003	15001
09997	15723
09998	17730
09999	13740
10000	09997
10001	01227
15000	10000
15001	10001
15002	09999
15003	01001
19999	

e di avere un file allocato a partire dal blocco fisico numero 01003. Se il disco ha 2 superfici, 100 cilindri per superficie e 100 settori per cilindro, quale è l'indirizzo fisico dei seguenti blocchi logici? Tenete presente che l'indirizzo fisico è espresso nel formato <superficie - cilindro - settore> e che tutti gli indirizzi partono da zero.

- a) 0 b) 2 c) 3

Risposta:

- a) <0, 10, 03> b) <1, 00, 01> c) <0, 12, 27>

ESERCIZIO 23

Supponete di avere un disco suddiviso in 5000 cilindri (da 0 a 4999) e che la testina sia posizionata sul cilindro 230. Supponete che la coda delle richieste sia la seguente:

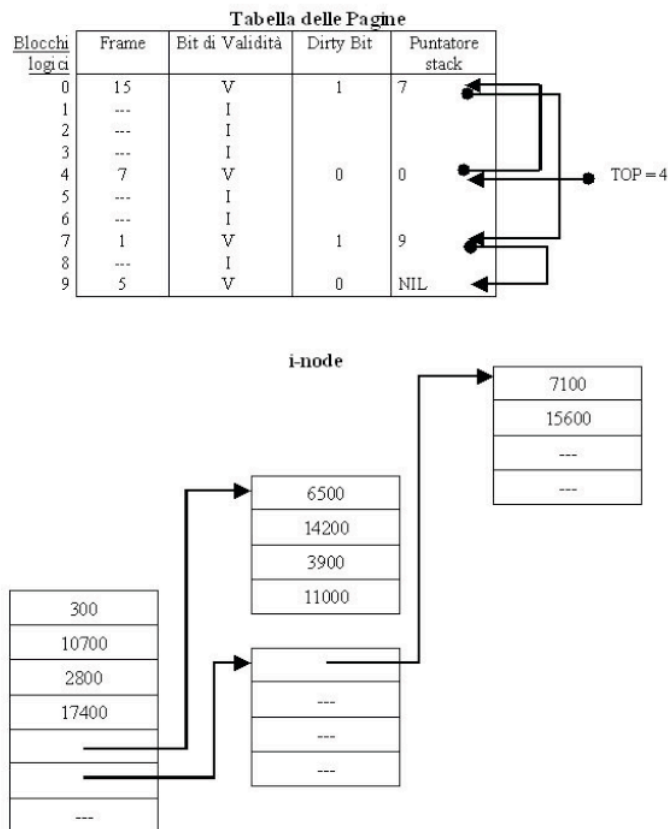
98, 1370, 1290, 1845, 2590, 970, 3400, 2740, 320.

Calcolare quale è la distanza totale percorsa dalla testina nei seguenti casi:

- a) viene adottato un algoritmo FCFS,
- b) viene adottato un algoritmo SSTF,
- c) viene adottato un algoritmo SCAN,
- d) viene adottato un algoritmo C-SCAN,
- e) viene adottato un algoritmo LOOK,
- f) viene adottato un algoritmo C-LOOK.

ESERCIZIO 24

Supponete di lavorare con un file system che adotta l'allocazione basata su i-node (tipo quella adottata da Linux) e come metodo di caching adotta la paginazione su richiesta (le pagine hanno dimensione pari ad un blocco logico di 1000 byte) con algoritmo di sostituzione LRU (realizzato tramite uno stack). Supponete che il file system risieda su un disco con 2 superfici, 100 cilindri per superficie, 100 settori per traccia, ed un settore per ogni blocco fisico. Supponete che l'algoritmo di scheduling del disco sia FCFS e che la testina del disco inizialmente sia posizionata sul cilindro 50. Supponete infine di avere un file a cui sono stati assegnati 4 frame e che ha la tabella delle pagine e l'i-node rappresentati in figura:



Data la seguente sequenza di operazioni sul file (nel formato numero OPERAZIONE - N° BLOCCO LOGICO):

- a) WRITE 3
- b) WRITE 4
- c) READ 3
- d) WRITE 8
- e) WRITE 5
- f) READ 4
- g) READ 3
- h) WRITE 6

Determinare la sequenza di frame coinvolti nelle precedenti operazioni ed il numero di cilindri attraversati (ad ogni operazione di WRITE il dirty bit del blocco viene messo ad 1).

ESERCIZIO 25

Supponete di lavorare con un sistema che adotta la paginazione su richiesta (le pagine hanno dimensione pari a 1000 byte) con algoritmo di sostituzione dell'orologio. Dato un processo a cui sono stati assegnati 4 frame e che ha la seguente tabella delle pagine:

Pagina	Frame	Bit di Validità	Bit di Riferimento	Ptr. lista circolare
0	7	V	1	→
1	---	I		
2	3	V	0	
3	---	I		
4	---	I		
5	---	I		
6	9	V	1	
7	---	I		
8	---	I		
9	---	I		
10	---	I		
11	---	I		
12	1	V	0	

Dati i seguenti indirizzi logici (formato: numero pag. - offset):

- a) 6 - 740
- b) 2 - 345
- c) 9 - 470
- d) 11 - 560
- e) 7 - 345
- f) 0 - 470
- g) 1 - 740
- h) 2 - 560
- i) 6 - 800
- l) 7 - 700

Trovare i corrispondenti indirizzi fisici. Tenete presente che ogni volta che si verifica un page fault bisogna sostituire una pagina in memoria con quella richiesta, che inizialmente il puntatore alla lista circolare è posizionato sulla pagina logica 0 e che ogni volta che viene caricata in memoria una pagina il bit di riferimento viene posto a 0.